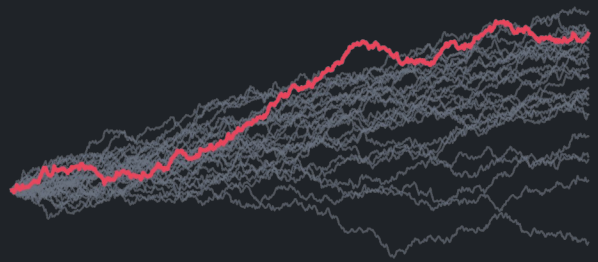
CMSC 838B · STUDENT LECTURE · FALL 2026

Backprop Through Continuous Time

Neural ODEs, neural SDEs, and the diffusion models built on them



Nishad Wajge

University of Maryland

ROADMAP

Four Parts, One Question

1

Continuous depth

Neural ODEs

2

Gradients without memory

The adjoint method

3

Add noise

Neural SDEs

4

Use and limits

Diffusion, applications, critique

The question all the way through: how do we backprop through a solver?

A ResNet Is Euler's Method

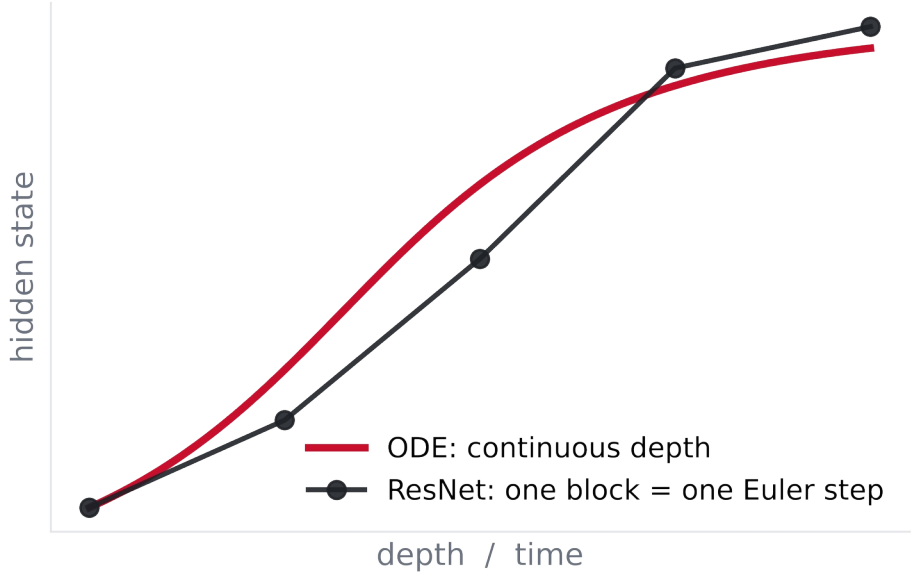
ResNet block

$$h_{k+1} = h_k + f_\theta(h_k)$$

step size → 0

Neural ODE

$$\frac{dh}{dt} = f_\theta(h, t)$$



Shrink the step and a deep network becomes a differential equation.

The Network Defines the Velocity

$$z(T) = z(0) + \int_0^T f_\theta(z(t),\ t)\, dt$$

fθ

any neural net

Solver

Euler, RK4, adaptive

Output

the state at time T

Training needs **dL / dθ** through the solver.

The forward pass is a call to an ODE solver.

Two Ways to Differentiate a Solver

Discretize, then differentiate

Backprop through every solver step

Memory	grows with steps
Gradient	exact for the numerics

Differentiate, then discretize

Solve a second ODE backward: the adjoint

Memory	constant
Gradient	approximate

Memory vs. gradient accuracy: the trade-off behind the rest of this talk.

The Adjoint Method

sensitivity

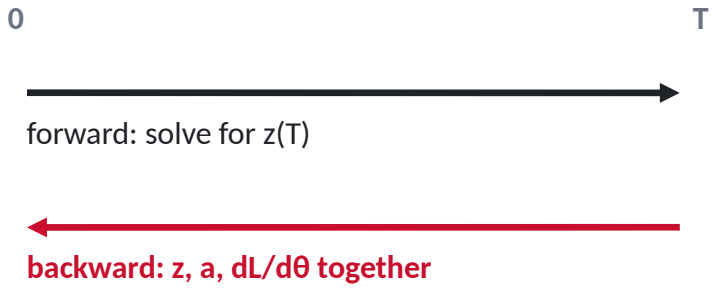
$$a(t) = \frac{\partial L}{\partial z(t)}$$

flows backward

$$\frac{da}{dt} = -a(t)^\top \frac{\partial f_\theta}{\partial z}$$

collects the gradient

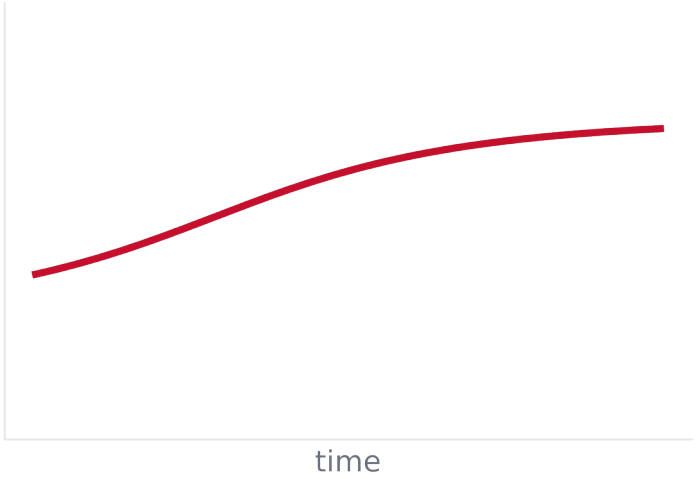
$$\frac{dL}{d\theta} = \int_0^T a(t)^\top \frac{\partial f_\theta}{\partial \theta}\, dt$$



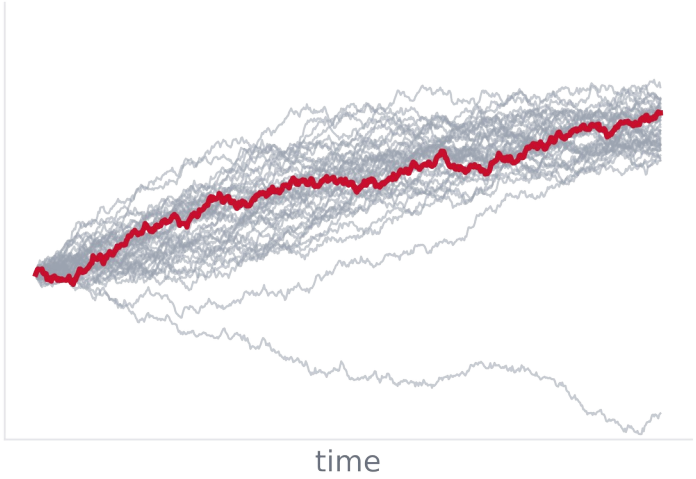
Each step needs $a^\intercal\, \partial f/\partial z$
one vector-Jacobian product: reverse-mode AD from Lecture 4

The adjoint is backprop, run as an ODE in reverse time.

One Future, or Many?



ODE: one deterministic path



SDE: a distribution over paths

When the system is random, a single path is the wrong answer.

A Neural SDE Is Two Networks

$$dZ_t = \mu_\theta(Z_t, t)\,dt \;+\; \sigma_\theta(Z_t, t)\,dW_t$$

Drift μθ

where the state is heading

Diffusion σθ

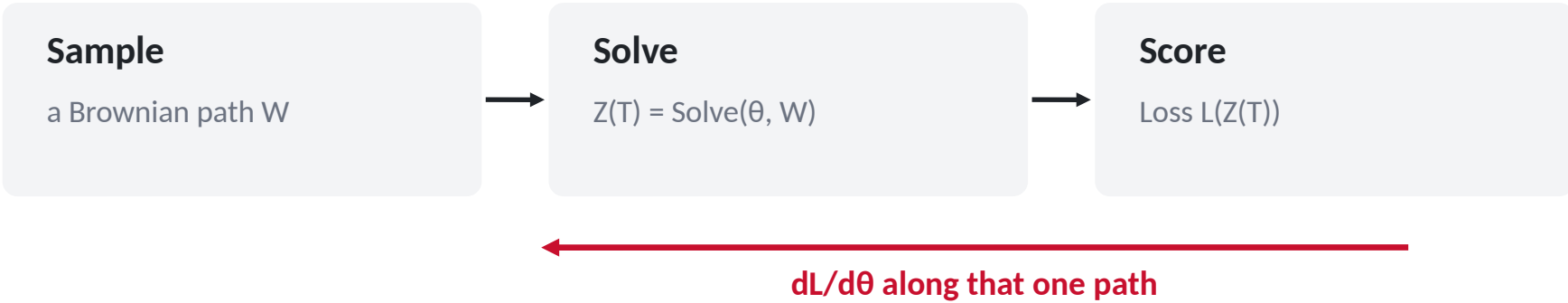
how hard the noise pushes

Simulate it (Euler–Maruyama)

$$Z_{k+1} = Z_k + \mu_\theta\,\Delta t + \sigma_\theta\,\sqrt{\Delta t}\;\varepsilon_k, \qquad \varepsilon_k \sim \mathcal{N}(0, I)$$

A neural SDE is a neural ODE that rolls dice at every step.

Fix the Noise, Then Differentiate



Same trick as the VAE's reparameterization: the randomness becomes an input.

With the noise held fixed, an SDE solve is just another differentiable function.

The Stochastic Adjoint: Two Catches



The backward pass must replay this exact path.

1

Same noise, backward

Store the path, or regenerate it on demand: Brownian tree / Brownian Interval

2

Different calculus

The backward SDE is Stratonovich; Itô needs a correction term

Fix: reversible Heun + its own adjoint → exact gradients, constant memory

Randomness makes the adjoint harder, not impossible.

In Code: About Ten Lines of JAX

```
def loss_fn(params, key):
    bm    = dfx.VirtualBrownianTree(0., 1., tol=1e-3, shape=(d,), key=key)
    terms = dfx.MultiTerm(dfx.ODETerm(drift),              # μθ dt
                          dfx.ControlTerm(diffusion, bm)) # σθ dW
    sol   = dfx.diffeqsolve(terms, dfx.ReversibleHeun(), 0., 1., dt0=0.01,
                            y0=z0, args=params,
                            adjoint=dfx.BacksolveAdjoint())  # O(1) memory
    return objective(sol.ys[-1])

grads = jax.grad(loss_fn)(params, key)
```

Measured: adjoint ≈ backprop to ~10^{-5}, and the gap shrinks as steps increase.

Three Ways to Train One

1

Latent SDE

Fit noisy time series

ELBO with a KL between two SDEs

Li et al., 2020

2

SDE-GAN

Match a distribution of paths

Neural CDE as the discriminator

Kidger et al., 2021

3

Calibration

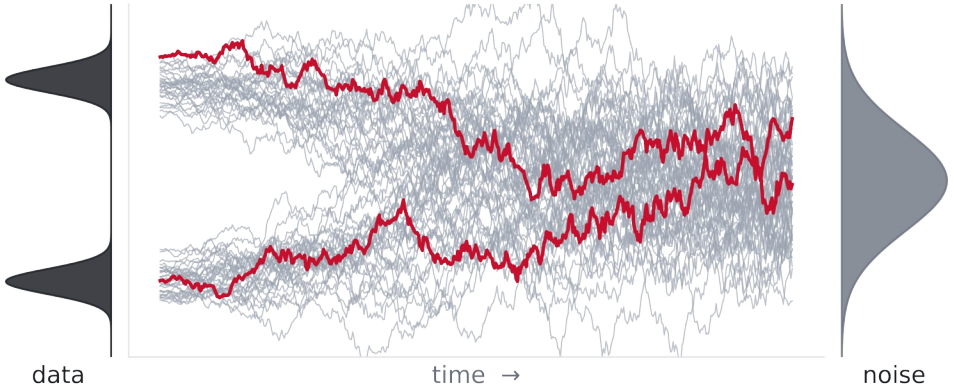
Match observed prices

Loss on model vs. market quantities

Gierjatowicz et al., 2020

Same differentiable solver. The loss decides what "fit" means.

Diffusion Models Are Neural SDEs



Forward: data → noise. Sampling runs it backward.

Forward: add noise

$$dx = f(x,t)\,dt + g(t)\,dW$$

Reverse: learned score

$$dx = \left[f(x,t) - g(t)^2\, s_\theta(x,t)\right]dt + g(t)\,d\bar{W}$$

Probability-flow ODE: same marginals

$$\frac{dx}{dt} = f(x,t) - \tfrac{1}{2}\, g(t)^2\, s_\theta(x,t)$$

$$s_\theta(x,t) \approx \nabla_x \log p_t(x)$$

Every diffusion or flow-matching model samples by solving a neural SDE or ODE.

Where Neural SDEs Show Up

Generative models

Images, audio, video

Song et al., 2021

Weather ensembles

GenCast: diffusion-sampled forecasts

Price et al., 2025

Robot policies

Diffusion Policy; π0 flow matching

Chi et al., 2023; Black et al., 2024

Quant finance

Calibrated, arbitrage-free market models

Gierjatowicz 2020; Cohen 2021

Irregular time series

Sparse, noisy sensor or clinical data

Li et al., 2020

Uncertainty

Bayesian nets as SDEs over weights

Xu et al., 2022

Anywhere the answer is a distribution over trajectories.

Where It Breaks

1	**Adjoint gradients drift**	The backward re-solve doesn't retrace the forward path	*Gholami et al., 2019*
2	**Solvers slow down in training**	Learned dynamics stiffen, step counts climb	*Finlay et al., 2020*
3	**The adjoint isn't cheap here**	Measured: flat in steps but ~150 MB; ~60× slower than backprop	*This repo*
4	**Training is fragile**	SDE-GANs need Lipschitz control; Itô/Stratonovich bugs are silent	*Kidger et al., 2021*

Constant memory is real. Exact gradients need the right solver.

DISCUSSION

Three Questions

1. When is the adjoint's memory saving worth its gradient error? Where in your own project?

2. Diffusion models fix σ and learn only the score. What do we lose by not learning the noise?

3. Could a differentiable simulator from this class become stochastic? What happens to its gradients?

Code and benchmarks: github.com/nishadw/neural-sde

Key References

Chen, Rubanova, Bettencourt, Duvenaud. Neural Ordinary Differential Equations. NeurIPS 2018.

Li, Wong, Chen, Duvenaud. Scalable Gradients for Stochastic Differential Equations. AISTATS 2020.

Kidger, Foster, Li, Oberhauser, Lyons. Neural SDEs as Infinite-Dimensional GANs. ICML 2021.

Kidger, Foster, Li, Lyons. Efficient and Accurate Gradients for Neural SDEs. NeurIPS 2021.

Kidger. On Neural Differential Equations. PhD thesis, Oxford, 2022.

Song et al. Score-Based Generative Modeling through SDEs. ICLR 2021.

Lipman et al. Flow Matching for Generative Modeling. ICLR 2023.

Gholami, Keutzer, Biros. ANODE. IJCAI 2019.

Finlay et al. How to Train Your Neural ODE. ICML 2020.

Gierjatowicz et al. Robust Pricing and Hedging via Neural SDEs. J. Comp. Finance, 2022.

Cohen, Reisinger, Wang. Arbitrage-Free Neural-SDE Market Models. 2021.

Price et al. Probabilistic Weather Forecasting with ML (GenCast). Nature 2025.

Chi et al. Diffusion Policy. RSS 2023. Black et al. π0. 2024.

Xu, Chen, Li, Duvenaud. Infinitely Deep Bayesian NNs with SDEs. AISTATS 2022.